



PROYECTO INTERMODULAR

BBDD



DIEGO GOMEZ ARCAS Y RUBEN CALVO MURIEDAS

Contenido

INTRODUCCION ¡Error! Marcador no definido.

DESARROLLO 3

 CODIGO COMPLETO 3

 TABLAS..... 11

 INSERTS 13

 CONSULTAS 16

IMÁGENES 26

..... 26

CONCLUSION 29

DESCRIPCION FUNCIONAL

SplitClass es una aplicación de escritorio desarrollada en JavaFX que permite gestionar gastos compartidos entre grupos de personas, similar a Tricount.

El usuario se registra e inicia sesión en la aplicación. Una vez dentro puede crear grupos y añadir miembros. Dentro de cada grupo se registran los gastos indicando quién pagó, cuánto costó y de qué categoría es. La aplicación calcula automáticamente cuánto debe pagar cada miembro. Cuando alguien salda su deuda se registra un pago entre los usuarios implicados. Además, la aplicación cuenta con un historial de movimientos y estadísticas con gráficos de gastos por mes y por categoría.

La interfaz está construida con JavaFX y FXML, los datos se almacenan en MySQL y la comunicación con la base de datos se realiza mediante JDBC con el patrón DAO.

DESARROLLO

CODIGO COMPLETO

```
DROP DATABASE IF EXISTS SplitClass;
CREATE DATABASE IF NOT EXISTS SplitClass;
USE SplitClass;

CREATE TABLE idiomas (
    id_idioma INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    codigo VARCHAR(10) NOT NULL UNIQUE
);

CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    cod_usuario VARCHAR(20) UNIQUE NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    telefono VARCHAR(20),
    id_idioma INT,
    alias VARCHAR(50),
    iban VARCHAR(34),
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_nacimiento DATE,
    FOREIGN KEY (id_idioma) REFERENCES idiomas(id_idioma) ON DELETE SET
NULL
);

CREATE TABLE categorias (
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,
    cod_categoria VARCHAR(20) UNIQUE NOT NULL,
    nombre VARCHAR(50) NOT NULL,
    icono VARCHAR(50)
);

CREATE TABLE grupos (
    id_grupo INT AUTO_INCREMENT PRIMARY KEY,
    cod_grupo VARCHAR(20) UNIQUE NOT NULL,
    titulo VARCHAR(100) NOT NULL,
    descripcion TEXT,
    moneda VARCHAR(10) DEFAULT 'EUR',
    fecha_creacion DATE NOT NULL,
    fecha_eliminacion DATE DEFAULT NULL
);

CREATE TABLE miembros_grupo (
```

```

    id_miembros INT AUTO_INCREMENT PRIMARY KEY,
    cod_miembros_grupo VARCHAR(20) UNIQUE NOT NULL,
    id_usuario INT NOT NULL,
    id_grupo INT NOT NULL,
    rol VARCHAR(20) DEFAULT 'miembro',
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE
CASCADE,
    FOREIGN KEY (id_grupo) REFERENCES grupos(id_grupo) ON DELETE CASCADE
);

CREATE TABLE gastos (
    id_gasto INT AUTO_INCREMENT PRIMARY KEY,
    cod_gasto VARCHAR(20) UNIQUE NOT NULL,
    concepto VARCHAR(150) NOT NULL,
    monto_total DOUBLE NOT NULL,
    fecha DATE NOT NULL,
    id_grupo INT NOT NULL,
    id_categoria INT,
    id_usuario_pagador INT NOT NULL,
    FOREIGN KEY (id_grupo) REFERENCES grupos(id_grupo) ON DELETE CASCADE,
    FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria) ON
DELETE SET NULL,
    FOREIGN KEY (id_usuario_pagador) REFERENCES usuarios(id_usuario)
);

CREATE TABLE reparto_gasto (
    id_reparto_gasto INT AUTO_INCREMENT PRIMARY KEY,
    cod_reparto_gasto VARCHAR(20) UNIQUE NOT NULL,
    id_gasto INT NOT NULL,
    id_usuario INT NOT NULL,
    cuota_deuda DOUBLE NOT NULL,
    FOREIGN KEY (id_gasto) REFERENCES gastos(id_gasto) ON DELETE CASCADE,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE
CASCADE
);

CREATE TABLE liquidaciones (
    id_liquidacion INT AUTO_INCREMENT PRIMARY KEY,
    cod_liquidacion VARCHAR(20) UNIQUE NOT NULL,
    monto DOUBLE NOT NULL,
    fecha_movimiento DATE NOT NULL,
    concepto VARCHAR(150),
    estado TINYINT(1) DEFAULT 0,
    id_emisor INT NOT NULL,
    id_receptor INT NOT NULL,
    id_grupo INT NOT NULL,
    FOREIGN KEY (id_emisor) REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_receptor) REFERENCES usuarios(id_usuario),
    FOREIGN KEY (id_grupo) REFERENCES grupos(id_grupo) ON DELETE CASCADE
);

```

```
);
```

```
INSERT INTO idiomas (nombre, codigo) VALUES
('Español', 'es'), ('Inglés', 'en'), ('Francés', 'fr'), ('Alemán', 'de'),
('Italiano', 'it'),
('Portugués', 'pt'), ('Chino', 'zh'), ('Japonés', 'ja'), ('Ruso', 'ru'),
('Árabe', 'ar');
```

```
INSERT INTO categorias (cod_categoria, nombre, icono) VALUES
('CAT001', 'Comida', 'ic_food'), ('CAT002', 'Transporte',
'ic_transport'),
('CAT003', 'Alojamiento', 'ic_hotel'), ('CAT004', 'Ocio', 'ic_leisure'),
('CAT005', 'Otros', 'ic_other');
```

```
INSERT INTO usuarios (cod_usuario, nombre, email, password, id_idioma,
alias, iban, fecha_nacimiento) VALUES
('USR001', 'Alejandro García', 'aaa@email.com', '1234', 1, 'Ale',
'ES1234567890123456789012', '1990-05-15'),
('USR002', 'Beatriz López', 'beatriz@email.com', 'hash_pw_2', 1, 'Bea',
'ES9876543210987654321098', '1992-08-22'),
('USR003', 'Carlos Martínez', 'carlos@email.com', 'hash_pw_3', 2,
'Carlitos', NULL, '1988-11-02'),
('USR004', 'Diana Pérez', 'diana@email.com', 'hash_pw_4', 1, 'Di',
'ES1122334455667788990011', '1995-01-30'),
('USR005', 'Eduardo Rodríguez', 'edu@email.com', 'hash_pw_5', 1, 'Edu',
NULL, '1993-04-12'),
('USR006', 'Fernanda Ruiz', 'fer@email.com', 'hash_pw_6', 1, 'Fer',
'ES2233445566778899001122', '1991-07-19'),
('USR007', 'Gabriel Sánchez', 'gabriel@email.com', 'hash_pw_7', 2,
'Gabi', NULL, '1989-12-05'),
('USR008', 'Helena Castro', 'helena@email.com', 'hash_pw_8', 1, 'Hel',
'ES3344556677889900112233', '1994-03-25'),
('USR009', 'Iván Torres', 'ivan@email.com', 'hash_pw_9', 1, 'Iván', NULL,
'1996-09-08'),
('USR010', 'Julia Morales', 'julia@email.com', 'hash_pw_10', 1, 'Juli',
'ES4455667788990011223344', '1992-02-14'),
('USR011', 'Kevin White', 'kevin@email.com', 'hash_pw_11', 2, 'Kev',
NULL, '1990-10-10'),
('USR012', 'Laura Vidal', 'laura@email.com', 'hash_pw_12', 1, 'Lau',
'ES5566778899001122334455', '1991-06-01'),
('USR013', 'Mario Gomez', 'mario@email.com', 'hash_pw_13', 1, 'Mario',
NULL, '1987-04-20'),
('USR014', 'Natalia Ortiz', 'natalia@email.com', 'hash_pw_14', 1, 'Nati',
'ES6677889900112233445566', '1998-12-31'),
('USR015', 'Oscar León', 'oscar@email.com', 'hash_pw_15', 1, 'Oscar',
NULL, '1993-05-15'),
('USR016', 'Patricia Ramos', 'patricia@email.com', 'hash_pw_16', 1,
'Patri', 'ES7788990011223344556677', '1994-08-11'),
```

```
(
'USR017', 'Quique Domínguez', 'quique@email.com', 'hash_pw_17', 1,
'Quique', NULL, '1990-01-05'),
('USR018', 'Rosa Méndez', 'rosa@email.com', 'hash_pw_18', 1, 'Rosa',
'ES8899001122334455667788', '1992-11-20'),
('USR019', 'Sergio Blanes', 'sergio@email.com', 'hash_pw_19', 1, 'Sergi',
NULL, '1989-03-30'),
('USR020', 'Teresa Soler', 'teresa@email.com', 'hash_pw_20', 1, 'Tere',
'ES9900112233445566778899', '1995-07-07'),
('USR021', 'Urbano Lima', 'urbano@email.com', 'hash_pw_21', 6, 'Urbi',
NULL, '1988-09-12'),
('USR022', 'Valeria Vega', 'valeria@email.com', 'hash_pw_22', 1, 'Val',
'ES0011223344556677889900', '1996-04-22'),
('USR023', 'Walter Smith', 'walter@email.com', 'hash_pw_23', 2, 'Walt',
NULL, '1991-12-15'),
('USR024', 'Ximena Gil', 'ximena@email.com', 'hash_pw_24', 1, 'Xime',
'ES110011001100110011', '1994-02-28'),
('USR025', 'Yago Costa', 'yago@email.com', 'hash_pw_25', 1, 'Yago', NULL,
'1993-10-05'),
('USR026', 'Zoe Herrero', 'zoe@email.com', 'hash_pw_26', 1, 'Zoe',
'ES220022002200220022', '1997-06-18'),
('USR027', 'Adrian Fluxá', 'adrian@email.com', 'hash_pw_27', 1, 'Adri',
NULL, '1992-01-12'),
('USR028', 'Bárbara Cano', 'barbara@email.com', 'hash_pw_28', 1, 'Barbi',
'ES330033003300330033', '1995-03-03'),
('USR029', 'Cristian Sol', 'cristian@email.com', 'hash_pw_29', 1, 'Cris',
NULL, '1990-11-11'),
('USR030', 'Daniela Paz', 'daniela@email.com', 'hash_pw_30', 1, 'Dani',
'ES440044004400440044', '1994-09-25');
```

```
INSERT INTO grupos (cod_grupo, titulo, descripcion, moneda,
fecha_creacion) VALUES
('GRP001', 'Viaje Cantabria', 'Viaje de fin de curso del grupo A', 'EUR',
'2025-01-10'),
('GRP002', 'Piso compartido', 'Gastos del piso de estudiantes', 'EUR',
'2025-01-15'),
('GRP003', 'Erasmus Berlín', 'Intercambio universitario Berlín', 'EUR',
'2025-02-01'),
('GRP004', 'Feria de Sevilla', 'Viaje semana feria', 'EUR', '2025-03-
20'),
('GRP005', 'Cumple Ana', 'Organización cumpleaños sorpresa', 'EUR',
'2025-04-05'),
('GRP006', 'Road trip Portugal', 'Viaje en coche por Portugal', 'EUR',
'2025-05-10'),
('GRP007', 'Cena navidad', 'Cena de empresa navideña', 'EUR', '2025-11-
20'),
('GRP008', 'Esquí Andorra', 'Fin de semana en la nieve', 'EUR', '2025-12-
01');
```

```

INSERT INTO miembros_grupo (cod_miembros_grupo, id_usuario, id_grupo,
rol) VALUES
('MBR001', 1, 1, 'admin'), ('MBR002', 2, 1, 'miembro'), ('MBR003', 3, 1,
'miembro'), ('MBR004', 4, 1, 'miembro'),
('MBR005', 5, 2, 'admin'), ('MBR006', 6, 2, 'miembro'), ('MBR007', 7, 2,
'miembro'),
('MBR008', 8, 3, 'admin'), ('MBR009', 9, 3, 'miembro'), ('MBR010', 10, 3,
'miembro'), ('MBR011', 11, 3, 'miembro'),
('MBR012', 12, 4, 'admin'), ('MBR013', 13, 4, 'miembro'), ('MBR014', 14,
4, 'miembro'),
('MBR015', 15, 5, 'admin'), ('MBR016', 16, 5, 'miembro');

```

```

INSERT INTO gastos (cod_gasto, concepto, monto_total, fecha, id_grupo,
id_categoria, id_usuario_pagador) VALUES
('GST001', 'Cena restaurante', 60.00, '2025-06-01', 1, 1, 1),
('GST002', 'Gasolina ida', 45.00, '2025-06-02', 1, 2, 2),
('GST003', 'Alquiler apartamento', 320.00, '2025-06-01', 1, 3, 3),
('GST004', 'Supermercado marzo', 87.50, '2025-03-05', 2, 1, 5),
('GST005', 'Factura luz febrero', 65.20, '2025-02-28', 2, 5, 6),
('GST006', 'Internet mensual', 30.00, '2025-03-01', 2, 5, 7),
('GST007', 'Vuelo Berlín', 210.00, '2025-09-10', 3, 2, 8),
('GST008', 'Hotel 3 noches', 270.00, '2025-09-11', 3, 3, 9),
('GST009', 'Entradas feria', 48.00, '2025-04-15', 4, 4, 12),
('GST010', 'Alquiler coche', 150.00, '2025-04-14', 4, 2, 13);

```

```

INSERT INTO reparto_gasto (cod_reparto_gasto, id_gasto, id_usuario,
cuota_deuda) VALUES
('RPT001', 1, 1, 15.00), ('RPT002', 1, 2, 15.00), ('RPT003', 1, 3,
15.00), ('RPT004', 1, 4, 15.00),
('RPT005', 2, 1, 22.50), ('RPT006', 2, 2, 22.50),
('RPT007', 4, 5, 29.17), ('RPT008', 4, 6, 29.17), ('RPT009', 4, 7,
29.16),
('RPT010', 5, 5, 21.73), ('RPT011', 5, 6, 21.73), ('RPT012', 5, 7,
21.74),
('RPT013', 7, 8, 70.00), ('RPT014', 7, 9, 70.00);

```

```

INSERT INTO liquidaciones (cod_liquidacion, monto, fecha_movimiento,
concepto, estado, id_emisor, id_receptor, id_grupo) VALUES
('LIQ001', 15.00, '2025-06-05', 'Pago cena Cantabria', 1, 2, 1, 1),
('LIQ002', 15.00, '2025-06-06', 'Pago cena Cantabria', 1, 3, 1, 1),
('LIQ003', 15.00, '2025-06-07', 'Pago cena Cantabria', 0, 4, 1, 1),
('LIQ004', 29.17, '2025-03-10', 'Pago supermercado marzo', 0, 6, 5, 2),
('LIQ005', 29.16, '2025-03-11', 'Pago supermercado marzo', 0, 7, 5, 2),
('LIQ006', 70.00, '2025-09-15', 'Pago vuelo Berlín', 0, 9, 8, 3);

```

```

-- =====
-- CONSULTAS
-- =====

```



```

-- 1. Todos los gastos de un grupo con el nombre del pagador y la categoría
-- Útil para mostrar el listado de gastos en la pantalla de detalle de grupo
SELECT
    g.concepto,
    g.monto_total,
    g.fecha,
    u.nombre AS pagador,
    c.nombre AS categoria
FROM gastos g
JOIN usuarios u ON g.id_usuario_pagador = u.id_usuario
JOIN categorias c ON g.id_categoria = c.id_categoria
WHERE g.id_grupo = 1
ORDER BY g.fecha DESC;

-- 2. Deuda total pendiente de cada usuario en un grupo
-- Útil para el panel de balances: cuánto debe cada miembro en total
SELECT
    u.nombre,
    SUM(rg.cuota_deuda) AS total_deuda
FROM reparto_gasto rg
JOIN usuarios u ON rg.id_usuario = u.id_usuario
JOIN gastos g ON rg.id_gasto = g.id_gasto
WHERE g.id_grupo = 1
GROUP BY u.id_usuario, u.nombre
ORDER BY total_deuda DESC;

-- 3. Liquidaciones pendientes de un grupo con nombre de emisor y receptor
-- Útil para mostrar quién debe pagar a quién y cuánto
SELECT
    e.nombre AS emisor,
    r.nombre AS receptor,
    l.monto,
    l.concepto,
    l.estado
FROM liquidaciones l
JOIN usuarios e ON l.id_emisor = e.id_usuario
JOIN usuarios r ON l.id_receptor = r.id_usuario
WHERE l.id_grupo = 1
    AND l.estado = 0;

-- 4. Gasto total por categoría en todos los grupos
-- Útil para la pantalla de estadísticas con el gráfico circular
SELECT
    c.nombre AS categoria,
    COUNT(g.id_gasto) AS num_gastos,

```

```

        SUM(g.monto_total) AS total_gastado
FROM gastos g
JOIN categorias c ON g.id_categoria = c.id_categoria
GROUP BY c.id_categoria, c.nombre
ORDER BY total_gastado DESC;

-- 5. Grupos a los que pertenece un usuario con su rol
-- Útil para mostrar la lista de grupos del usuario en la pantalla principal
SELECT
    gr.titulo,
    gr.descripcion,
    gr.moneda,
    mg.rol
FROM miembros_grupo mg
JOIN grupos gr ON mg.id_grupo = gr.id_grupo
WHERE mg.id_usuario = 1
    AND gr.fecha_eliminacion IS NULL
ORDER BY gr.fecha_creacion DESC;

-- 6. Usuarios que aún no han pagado su parte en un gasto concreto
-- Útil para recordatorios o marcar deudas pendientes
SELECT
    u.nombre,
    rg.cuota_deuda
FROM reparto_gasto rg
JOIN usuarios u ON rg.id_usuario = u.id_usuario
WHERE rg.id_gasto = 1
    AND rg.id_usuario NOT IN (
        SELECT id_emisor
        FROM liquidaciones
        WHERE id_grupo = 1
            AND estado = 1 -- 1 representa 'completado'
    );

-- 7. Gasto total por mes en un grupo (para el gráfico de barras mensual)
SELECT
    DATE_FORMAT(g.fecha, '%Y-%m') AS mes,
    SUM(g.monto_total) AS total_mes
FROM gastos g
WHERE g.id_grupo = 1
GROUP BY mes
ORDER BY mes ASC;

-- 8. Miembros de un grupo con el total que ha pagado cada uno
-- Útil para ver quién ha adelantado más dinero
SELECT
    u.nombre,
    COALESCE(SUM(g.monto_total), 0) AS total_pagado

```

```

FROM miembros_grupo mg
JOIN usuarios u ON mg.id_usuario = u.id_usuario
LEFT JOIN gastos g ON g.id_usuario_pagador = u.id_usuario
                    AND g.id_grupo = mg.id_grupo
WHERE mg.id_grupo = 1
GROUP BY u.id_usuario, u.nombre
ORDER BY total_pagado DESC;

-- 9. Balance neto de cada usuario en un grupo (pagado - debe)
-- Positivo = le deben dinero | Negativo = debe dinero
SELECT
    u.nombre,
    COALESCE(pagos.total_pagado, 0) AS total_pagado,
    COALESCE(deudas.total_debe, 0) AS total_debe,
    (COALESCE(pagos.total_pagado, 0) - COALESCE(deudas.total_debe, 0)) AS
balance_neto
FROM miembros_grupo mg
JOIN usuarios u ON mg.id_usuario = u.id_usuario
LEFT JOIN (
    SELECT id_usuario_pagador, id_grupo, SUM(monto_total) AS total_pagado
    FROM gastos
    GROUP BY id_usuario_pagador, id_grupo
) pagos ON pagos.id_usuario_pagador = u.id_usuario AND pagos.id_grupo =
mg.id_grupo
LEFT JOIN (
    SELECT rg.id_usuario, g.id_grupo, SUM(rg.cuota_deuda) AS total_debe
    FROM reparto_gasto rg
    JOIN gastos g ON rg.id_gasto = g.id_gasto
    GROUP BY rg.id_usuario, g.id_grupo
) deudas ON deudas.id_usuario = u.id_usuario AND deudas.id_grupo =
mg.id_grupo
WHERE mg.id_grupo = 1
ORDER BY balance_neto DESC;

SELECT
    'Gasto' AS tipo,
    g.concepto,
    g.monto_total AS importe,
    g.fecha AS fecha,
    gr.titulo AS grupo
FROM gastos g
JOIN grupos gr ON g.id_grupo = gr.id_grupo
WHERE g.id_usuario_pagador = 1

UNION ALL

SELECT
    'Liquidacion' AS tipo,

```

```

        l.concepto,
        l.monto AS importe,
        l.fecha_movimiento AS fecha,
        gr.titulo AS grupo
FROM liquidaciones l
JOIN grupos gr ON l.id_grupo = gr.id_grupo
WHERE l.id_emisor = 1

ORDER BY fecha DESC;

```

TABLAS

```

DROP DATABASE IF EXISTS SplitClass;
CREATE DATABASE IF NOT EXISTS SplitClass;
USE SplitClass;

CREATE TABLE IDIOMA (
    Codigo VARCHAR(5) NOT NULL,
    Nombre VARCHAR(50) NOT NULL,
    PRIMARY KEY (Codigo)
);

CREATE TABLE CATEGORIA (
    Id_categoria INT NOT NULL AUTO_INCREMENT,
    Nombre VARCHAR(50) NOT NULL,
    Icono VARCHAR(100),
    PRIMARY KEY (Id_categoria)
);

CREATE TABLE USUARIO (
    Id_Usuario INT NOT NULL AUTO_INCREMENT,
    Nombre VARCHAR(100) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    Password VARCHAR(255) NOT NULL,
    Idioma VARCHAR(5) DEFAULT 'es',
    Alias VARCHAR(50),
    IBAN VARCHAR(34),
    Fecha_Creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Fecha_Nacimiento DATE,
    PRIMARY KEY (Id_Usuario),
    CONSTRAINT fk_usuario_idioma FOREIGN KEY (Idioma) REFERENCES IDIOMA
    (Codigo)
);

CREATE TABLE GRUPO (
    Id_Grupo INT NOT NULL AUTO_INCREMENT,
    Titulo VARCHAR(100) NOT NULL,
    Descripcion TEXT,
    Moneda VARCHAR(10) DEFAULT 'EUR',

```

```

        Fecha_creacion    DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
        Fecha_eliminacion DATETIME,
        PRIMARY KEY (Id_Grupo)
    );

CREATE TABLE MIEMBROS_GRUPO (
    Id_Miembros_Grupo INT            NOT NULL AUTO_INCREMENT,
    Id_Usuario          INT            NOT NULL,
    Id_Grupo           INT            NOT NULL,
    Rol                 VARCHAR(20) NOT NULL DEFAULT 'miembro',
    PRIMARY KEY (Id_Miembros_Grupo),
    UNIQUE KEY uq_usuario_grupo (Id_Usuario, Id_Grupo),
    CONSTRAINT fk_mg_usuario FOREIGN KEY (Id_Usuario) REFERENCES USUARIO
(Id_Usuario),
    CONSTRAINT fk_mg_grupo    FOREIGN KEY (Id_Grupo)    REFERENCES
GRUPO (Id_Grupo)
);

CREATE TABLE GASTO (
    Id_Gasto            INT            NOT NULL AUTO_INCREMENT,
    Concepto            VARCHAR(200) NOT NULL,
    Monto_total         DECIMAL(10, 2) NOT NULL,
    Fecha               DATE           NOT NULL,
    Id_Grupo           INT            NOT NULL,
    Id_Categoria        INT,
    Id_Usuario_Pagador INT            NOT NULL,
    PRIMARY KEY (Id_Gasto),
    CONSTRAINT fk_gasto_grupo FOREIGN KEY (Id_Grupo) REFERENCES
GRUPO (Id_Grupo),
    CONSTRAINT fk_gasto_cat  FOREIGN KEY (Id_Categoria) REFERENCES
CATEGORIA (Id_categoria),
    CONSTRAINT fk_gasto_pagador FOREIGN KEY (Id_Usuario_Pagador) REFERENCES
USUARIO (Id_Usuario)
);

CREATE TABLE REPARTO_GASTO (
    ID_Reparto_Gasto INT            NOT NULL AUTO_INCREMENT,
    Id_Gasto          INT            NOT NULL,
    Id_Usuario        INT            NOT NULL,
    Cuota_deuda       DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (ID_Reparto_Gasto),
    UNIQUE KEY uq_reparto (Id_Gasto, Id_Usuario),
    CONSTRAINT fk_reparto_gasto FOREIGN KEY (Id_Gasto) REFERENCES
GASTO (Id_Gasto),
    CONSTRAINT fk_reparto_usuario FOREIGN KEY (Id_Usuario) REFERENCES
USUARIO (Id_Usuario)
);

CREATE TABLE LIQUIDACION (

```

```

    Id_liquidacion    INT                NOT NULL AUTO_INCREMENT,
    Monto              DECIMAL(10, 2) NOT NULL,
    Fecha_movimiento  DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Concepto           VARCHAR(200),
    Estado             VARCHAR(20)      NOT NULL DEFAULT 'pendiente',
    Id_Emisor          INT               NOT NULL,
    Id_Receptor        INT               NOT NULL,
    Id_Grupo          INT               NOT NULL,
    PRIMARY KEY (Id_liquidacion),
    CONSTRAINT fk_liq_emisor FOREIGN KEY (Id_Emisor) REFERENCES USUARIO
(Id_Usuario),
    CONSTRAINT fk_liq_receptor FOREIGN KEY (Id_Receptor) REFERENCES USUARIO
(Id_Usuario),
    CONSTRAINT fk_liq_grupo FOREIGN KEY (Id_Grupo) REFERENCES
GRUPO (Id_Grupo)
);

```

INSERTS

```

INSERT INTO idiomas (nombre, codigo) VALUES
('Español', 'es'), ('Inglés', 'en'), ('Francés', 'fr'), ('Alemán', 'de'),
('Italiano', 'it'),
('Portugués', 'pt'), ('Chino', 'zh'), ('Japonés', 'ja'), ('Ruso', 'ru'),
('Árabe', 'ar');

```

```

INSERT INTO categorias (cod_categoria, nombre, icono) VALUES
('CAT001', 'Comida', 'ic_food'), ('CAT002', 'Transporte',
'ic_transport'),
('CAT003', 'Alojamiento', 'ic_hotel'), ('CAT004', 'Ocio', 'ic_leisure'),
('CAT005', 'Otros', 'ic_other');

```

```

INSERT INTO usuarios (cod_usuario, nombre, email, password, id_idioma,
alias, iban, fecha_nacimiento) VALUES
('USR001', 'Alejandro García', 'aaa@email.com', '1234', 1, 'Ale',
'ES1234567890123456789012', '1990-05-15'),
('USR002', 'Beatriz López', 'beatriz@email.com', 'hash_pw_2', 1, 'Bea',
'ES9876543210987654321098', '1992-08-22'),
('USR003', 'Carlos Martínez', 'carlos@email.com', 'hash_pw_3', 2,
'Carlitos', NULL, '1988-11-02'),
('USR004', 'Diana Pérez', 'diana@email.com', 'hash_pw_4', 1, 'Di',
'ES1122334455667788990011', '1995-01-30'),
('USR005', 'Eduardo Rodríguez', 'edu@email.com', 'hash_pw_5', 1, 'Edu',
NULL, '1993-04-12'),
('USR006', 'Fernanda Ruiz', 'fer@email.com', 'hash_pw_6', 1, 'Fer',
'ES2233445566778899001122', '1991-07-19'),
('USR007', 'Gabriel Sánchez', 'gabriel@email.com', 'hash_pw_7', 2,
'Gabi', NULL, '1989-12-05'),
('USR008', 'Helena Castro', 'helena@email.com', 'hash_pw_8', 1, 'Hel',
'ES3344556677889900112233', '1994-03-25'),

```

```
(
    'USR009', 'Iván Torres', 'ivan@email.com', 'hash_pw_9', 1, 'Iván', NULL,
    '1996-09-08'),
    ('USR010', 'Julia Morales', 'julia@email.com', 'hash_pw_10', 1, 'Juli',
    'ES4455667788990011223344', '1992-02-14'),
    ('USR011', 'Kevin White', 'kevin@email.com', 'hash_pw_11', 2, 'Kev',
    NULL, '1990-10-10'),
    ('USR012', 'Laura Vidal', 'laura@email.com', 'hash_pw_12', 1, 'Lau',
    'ES5566778899001122334455', '1991-06-01'),
    ('USR013', 'Mario Gomez', 'mario@email.com', 'hash_pw_13', 1, 'Mario',
    NULL, '1987-04-20'),
    ('USR014', 'Natalia Ortiz', 'natalia@email.com', 'hash_pw_14', 1, 'Nati',
    'ES6677889900112233445566', '1998-12-31'),
    ('USR015', 'Oscar León', 'oscar@email.com', 'hash_pw_15', 1, 'Oscar',
    NULL, '1993-05-15'),
    ('USR016', 'Patricia Ramos', 'patricia@email.com', 'hash_pw_16', 1,
    'Patri', 'ES7788990011223344556677', '1994-08-11'),
    ('USR017', 'Quique Domínguez', 'quique@email.com', 'hash_pw_17', 1,
    'Quique', NULL, '1990-01-05'),
    ('USR018', 'Rosa Méndez', 'rosa@email.com', 'hash_pw_18', 1, 'Rosa',
    'ES8899001122334455667788', '1992-11-20'),
    ('USR019', 'Sergio Blanes', 'sergio@email.com', 'hash_pw_19', 1, 'Sergi',
    NULL, '1989-03-30'),
    ('USR020', 'Teresa Soler', 'teresa@email.com', 'hash_pw_20', 1, 'Tere',
    'ES9900112233445566778899', '1995-07-07'),
    ('USR021', 'Urbano Lima', 'urbano@email.com', 'hash_pw_21', 6, 'Urbi',
    NULL, '1988-09-12'),
    ('USR022', 'Valeria Vega', 'valeria@email.com', 'hash_pw_22', 1, 'Val',
    'ES0011223344556677889900', '1996-04-22'),
    ('USR023', 'Walter Smith', 'walter@email.com', 'hash_pw_23', 2, 'Walt',
    NULL, '1991-12-15'),
    ('USR024', 'Ximena Gil', 'ximena@email.com', 'hash_pw_24', 1, 'Xime',
    'ES110011001100110011', '1994-02-28'),
    ('USR025', 'Yago Costa', 'yago@email.com', 'hash_pw_25', 1, 'Yago', NULL,
    '1993-10-05'),
    ('USR026', 'Zoe Herrero', 'zoe@email.com', 'hash_pw_26', 1, 'Zoe',
    'ES2200220022002200220022', '1997-06-18'),
    ('USR027', 'Adrian Fluxá', 'adrian@email.com', 'hash_pw_27', 1, 'Adri',
    NULL, '1992-01-12'),
    ('USR028', 'Bárbara Cano', 'barbara@email.com', 'hash_pw_28', 1, 'Barbi',
    'ES3300330033003300330033', '1995-03-03'),
    ('USR029', 'Cristian Sol', 'cristian@email.com', 'hash_pw_29', 1, 'Cris',
    NULL, '1990-11-11'),
    ('USR030', 'Daniela Paz', 'daniela@email.com', 'hash_pw_30', 1, 'Dani',
    'ES4400440044004400440044', '1994-09-25');
```

```
INSERT INTO grupos (cod_grupo, titulo, descripcion, moneda,
fecha_creacion) VALUES
('GRP001', 'Viaje Cantabria', 'Viaje de fin de curso del grupo A', 'EUR',
'2025-01-10'),
```

```
( 'GRP002', 'Piso compartido', 'Gastos del piso de estudiantes', 'EUR',
'2025-01-15'),
( 'GRP003', 'Erasmus Berlín', 'Intercambio universitario Berlín', 'EUR',
'2025-02-01'),
( 'GRP004', 'Feria de Sevilla', 'Viaje semana feria', 'EUR', '2025-03-
20'),
( 'GRP005', 'Cumple Ana', 'Organización cumpleaños sorpresa', 'EUR',
'2025-04-05'),
( 'GRP006', 'Road trip Portugal', 'Viaje en coche por Portugal', 'EUR',
'2025-05-10'),
( 'GRP007', 'Cena navidad', 'Cena de empresa navideña', 'EUR', '2025-11-
20'),
( 'GRP008', 'Esquí Andorra', 'Fin de semana en la nieve', 'EUR', '2025-12-
01');
```

```
INSERT INTO miembros_grupo (cod_miembros_grupo, id_usuario, id_grupo,
rol) VALUES
( 'MBR001', 1, 1, 'admin'), ( 'MBR002', 2, 1, 'miembro'), ( 'MBR003', 3, 1,
'miembro'), ( 'MBR004', 4, 1, 'miembro'),
( 'MBR005', 5, 2, 'admin'), ( 'MBR006', 6, 2, 'miembro'), ( 'MBR007', 7, 2,
'miembro'),
( 'MBR008', 8, 3, 'admin'), ( 'MBR009', 9, 3, 'miembro'), ( 'MBR010', 10, 3,
'miembro'), ( 'MBR011', 11, 3, 'miembro'),
( 'MBR012', 12, 4, 'admin'), ( 'MBR013', 13, 4, 'miembro'), ( 'MBR014', 14,
4, 'miembro'),
( 'MBR015', 15, 5, 'admin'), ( 'MBR016', 16, 5, 'miembro');
```

```
INSERT INTO gastos (cod_gasto, concepto, monto_total, fecha, id_grupo,
id_categoria, id_usuario_pagador) VALUES
( 'GST001', 'Cena restaurante', 60.00, '2025-06-01', 1, 1, 1),
( 'GST002', 'Gasolina ida', 45.00, '2025-06-02', 1, 2, 2),
( 'GST003', 'Alquiler apartamento', 320.00, '2025-06-01', 1, 3, 3),
( 'GST004', 'Supermercado marzo', 87.50, '2025-03-05', 2, 1, 5),
( 'GST005', 'Factura luz febrero', 65.20, '2025-02-28', 2, 5, 6),
( 'GST006', 'Internet mensual', 30.00, '2025-03-01', 2, 5, 7),
( 'GST007', 'Vuelo Berlín', 210.00, '2025-09-10', 3, 2, 8),
( 'GST008', 'Hotel 3 noches', 270.00, '2025-09-11', 3, 3, 9),
( 'GST009', 'Entradas feria', 48.00, '2025-04-15', 4, 4, 12),
( 'GST010', 'Alquiler coche', 150.00, '2025-04-14', 4, 2, 13);
```

```
INSERT INTO reparto_gasto (cod_reparto_gasto, id_gasto, id_usuario,
cuota_deuda) VALUES
( 'RPT001', 1, 1, 15.00), ( 'RPT002', 1, 2, 15.00), ( 'RPT003', 1, 3,
15.00), ( 'RPT004', 1, 4, 15.00),
( 'RPT005', 2, 1, 22.50), ( 'RPT006', 2, 2, 22.50),
( 'RPT007', 4, 5, 29.17), ( 'RPT008', 4, 6, 29.17), ( 'RPT009', 4, 7,
29.16),
```



```

('RPT010', 5, 5, 21.73), ('RPT011', 5, 6, 21.73), ('RPT012', 5, 7,
21.74),
('RPT013', 7, 8, 70.00), ('RPT014', 7, 9, 70.00);

INSERT INTO liquidaciones (cod_liquidacion, monto, fecha_movimiento,
concepto, estado, id_emisor, id_receptor, id_grupo) VALUES
('LIQ001', 15.00, '2025-06-05', 'Pago cena Cantabria', 1, 2, 1, 1),
('LIQ002', 15.00, '2025-06-06', 'Pago cena Cantabria', 1, 3, 1, 1),
('LIQ003', 15.00, '2025-06-07', 'Pago cena Cantabria', 0, 4, 1, 1),
('LIQ004', 29.17, '2025-03-10', 'Pago supermercado marzo', 0, 6, 5, 2),
('LIQ005', 29.16, '2025-03-11', 'Pago supermercado marzo', 0, 7, 5, 2),
('LIQ006', 70.00, '2025-09-15', 'Pago vuelo Berlín', 0, 9, 8, 3);

```

CONSULTAS

```

-- =====
-- CONSULTAS
-- =====

-- 1. Todos los gastos de un grupo con el nombre del pagador y la
categoría
-- Útil para mostrar el listado de gastos en la pantalla de detalle de
grupo
SELECT
    g.concepto,
    g.monto_total,
    g.fecha,
    u.nombre AS pagador,
    c.nombre AS categoria
FROM gastos g
JOIN usuarios u ON g.id_usuario_pagador = u.id_usuario
JOIN categorias c ON g.id_categoria = c.id_categoria
WHERE g.id_grupo = 1
ORDER BY g.fecha DESC;

-- 2. Deuda total pendiente de cada usuario en un grupo
-- Útil para el panel de balances: cuánto debe cada miembro en total
SELECT
    u.nombre,
    SUM(rg.cuota_deuda) AS total_deuda
FROM reparto_gasto rg
JOIN usuarios u ON rg.id_usuario = u.id_usuario
JOIN gastos g ON rg.id_gasto = g.id_gasto
WHERE g.id_grupo = 1
GROUP BY u.id_usuario, u.nombre
ORDER BY total_deuda DESC;

```

```

-- 3. Liquidaciones pendientes de un grupo con nombre de emisor y receptor
-- Útil para mostrar quién debe pagar a quién y cuánto
SELECT
    e.nombre AS emisor,
    r.nombre AS receptor,
    l.monto,
    l.concepto,
    l.estado
FROM liquidaciones l
JOIN usuarios e ON l.id_emisor = e.id_usuario
JOIN usuarios r ON l.id_receptor = r.id_usuario
WHERE l.id_grupo = 1
    AND l.estado = 0;

-- 4. Gasto total por categoría en todos los grupos
-- Útil para la pantalla de estadísticas con el gráfico circular
SELECT
    c.nombre AS categoria,
    COUNT(g.id_gasto) AS num_gastos,
    SUM(g.monto_total) AS total_gastado
FROM gastos g
JOIN categorias c ON g.id_categoria = c.id_categoria
GROUP BY c.id_categoria, c.nombre
ORDER BY total_gastado DESC;

-- 5. Grupos a los que pertenece un usuario con su rol
-- Útil para mostrar la lista de grupos del usuario en la pantalla principal
SELECT
    gr.titulo,
    gr.descripcion,
    gr.moneda,
    mg.rol
FROM miembros_grupo mg
JOIN grupos gr ON mg.id_grupo = gr.id_grupo
WHERE mg.id_usuario = 1
    AND gr.fecha_eliminacion IS NULL
ORDER BY gr.fecha_creacion DESC;

-- 6. Usuarios que aún no han pagado su parte en un gasto concreto
-- Útil para recordatorios o marcar deudas pendientes
SELECT
    u.nombre,
    rg.cuota_deuda
FROM reparto_gasto rg
JOIN usuarios u ON rg.id_usuario = u.id_usuario
WHERE rg.id_gasto = 1
    AND rg.id_usuario NOT IN (

```

```

        SELECT id_emisor
        FROM liquidaciones
        WHERE id_grupo = 1
              AND estado = 1 -- 1 representa 'completado'
    );

-- 7. Gasto total por mes en un grupo (para el gráfico de barras mensual)
SELECT
    DATE_FORMAT(g.fecha, '%Y-%m') AS mes,
    SUM(g.monto_total) AS total_mes
FROM gastos g
WHERE g.id_grupo = 1
GROUP BY mes
ORDER BY mes ASC;

-- 8. Miembros de un grupo con el total que ha pagado cada uno
--     Útil para ver quién ha adelantado más dinero
SELECT
    u.nombre,
    COALESCE(SUM(g.monto_total), 0) AS total_pagado
FROM miembros_grupo mg
JOIN usuarios u ON mg.id_usuario = u.id_usuario
LEFT JOIN gastos g ON g.id_usuario_pagador = u.id_usuario
                  AND g.id_grupo = mg.id_grupo
WHERE mg.id_grupo = 1
GROUP BY u.id_usuario, u.nombre
ORDER BY total_pagado DESC;

-- 9. Balance neto de cada usuario en un grupo (pagado - debe)
--     Positivo = le deben dinero | Negativo = debe dinero
SELECT
    u.nombre,
    COALESCE(pagos.total_pagado, 0) AS total_pagado,
    COALESCE(deudas.total_debe, 0) AS total_debe,
    (COALESCE(pagos.total_pagado, 0) - COALESCE(deudas.total_debe, 0)) AS
balance_neto
FROM miembros_grupo mg
JOIN usuarios u ON mg.id_usuario = u.id_usuario
LEFT JOIN (
    SELECT id_usuario_pagador, id_grupo, SUM(monto_total) AS total_pagado
    FROM gastos
    GROUP BY id_usuario_pagador, id_grupo
) pagos ON pagos.id_usuario_pagador = u.id_usuario AND pagos.id_grupo =
mg.id_grupo
LEFT JOIN (
    SELECT rg.id_usuario, g.id_grupo, SUM(rg.cuota_deuda) AS total_debe
    FROM reparto_gasto rg
    JOIN gastos g ON rg.id_gasto = g.id_gasto
    GROUP BY rg.id_usuario, g.id_grupo

```

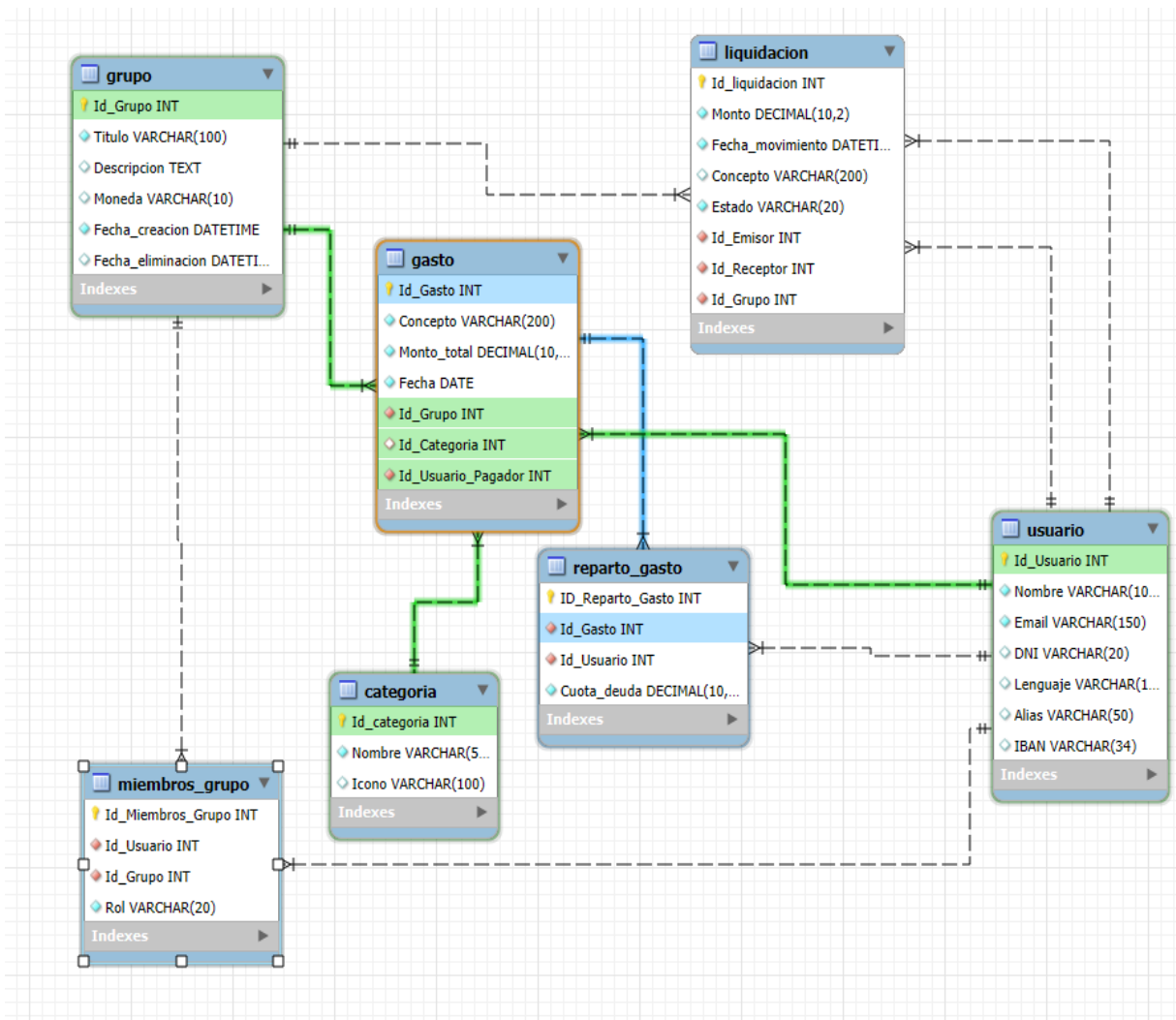
```
) deudas ON deudas.id_usuario = u.id_usuario AND deudas.id_grupo =  
mg.id_grupo  
WHERE mg.id_grupo = 1  
ORDER BY balance_neto DESC;
```

```
SELECT  
    'Gasto' AS tipo,  
    g.concepto,  
    g.monto_total AS importe,  
    g.fecha AS fecha,  
    gr.titulo AS grupo  
FROM gastos g  
JOIN grupos gr ON g.id_grupo = gr.id_grupo  
WHERE g.id_usuario_pagador = 1
```

```
UNION ALL
```

```
SELECT  
    'Liquidacion' AS tipo,  
    l.concepto,  
    l.monto AS importe,  
    l.fecha_movimiento AS fecha,  
    gr.titulo AS grupo  
FROM liquidaciones l  
JOIN grupos gr ON l.id_grupo = gr.id_grupo  
WHERE l.id_emisor = 1  
  
ORDER BY fecha DESC;
```

REVERSER ENGINEER



CONSULTAS 2.0

1. Mostrar todos los gastos de un grupo indicando el nombre del pagador y la categoría, ordenados del más reciente al más antiguo.

	Concepto	Monto_total	Fecha	Pagador	Categoría
▶	Gasolina ida	45.00	2025-06-02	Beatriz López	Transporte
	Cena restaurante	60.00	2025-06-01	Alejandro García	Comida
	Alquiler apartamento	320.00	2025-06-01	Carlos Martínez	Alojamiento

```
SELECT
    G.Concepto,
    G.Monto_total,
    G.Fecha,
    U.Nombre    AS Pagador,
    C.Nombre    AS Categoria
FROM GASTO G
JOIN USUARIO U ON G.Id Usuario Pagador = U.Id Usuario
```

```
JOIN CATEGORIA C ON G.Id_Categoria = C.Id_categoria
WHERE G.Id_Grupo = 1
ORDER BY G.Fecha DESC;
```

2. Calcular la deuda total acumulada de cada miembro dentro de un grupo, ordenada de mayor a menor.

	Nombre	Total_deuda
▶	Alejandro García	37.50
	Beatriz López	37.50
	Carlos Martínez	15.00
	Diana Pérez	15.00

```
SELECT
    U.Nombre,
    SUM(RG.Cuota_deuda) AS Total_deuda
FROM REPARTO_GASTO RG
JOIN USUARIO U ON RG.Id_Usuario = U.Id_Usuario
JOIN GASTO G ON RG.Id_Gasto = G.Id_Gasto
WHERE G.Id_Grupo = 1
GROUP BY U.Id_Usuario, U.Nombre
ORDER BY Total_deuda DESC;
```

3. Listar todas las liquidaciones pendientes de un grupo mostrando quién debe pagar, a quién y cuánto.

	Emisor	Receptor	Monto	Concepto	Estado
▶	Diana Pérez	Alejandro García	15.00	Pago cena Cantabria	pendiente

```
SELECT
    E.Nombre AS Emisor,
    R.Nombre AS Receptor,
    L.Monto,
    L.Concepto,
    L.Estado
FROM LIQUIDACION L
JOIN USUARIO E ON L.Id_Emisor = E.Id_Usuario
JOIN USUARIO R ON L.Id_Receptor = R.Id_Usuario
WHERE L.Id_Grupo = 1
    AND L.Estado = 'pendiente';
```

4. Obtener el gasto total y el número de gastos agrupados por categoría en todos los grupos.

	Categoria	Num_gastos	Total_gastado
▶	Alojamiento	2	590.00
	Transporte	3	405.00
	Comida	2	147.50
	Otros	2	95.20
	Ocio	1	48.00

```
SELECT
    C.Nombre                AS Categoria,
    COUNT(G.Id_Gasto)      AS Num_gastos,
    SUM(G.Monto_total)     AS Total_gastado
FROM GASTO G
JOIN CATEGORIA C ON G.Id_Categoria = C.Id_categoria
GROUP BY C.Id_categoria, C.Nombre
ORDER BY Total_gastado DESC;
```

5. Mostrar todos los grupos a los que pertenece un usuario junto con su rol, excluyendo los grupos eliminados.

	Titulo	Descripcion	Moneda	Rol
▶	Viaje Cantabria	Viaje de fin de curso del grupo A	EUR	admin

```
SELECT
    GR.Titulo,
    GR.Descripcion,
    GR.Moneda,
    MG.Rol
FROM MIEMBROS_GRUPO MG
JOIN GRUPO GR ON MG.Id_Grupo = GR.Id_Grupo
WHERE MG.Id_Usuario = 1
    AND GR.Fecha_eliminacion IS NULL
ORDER BY GR.Fecha_creacion DESC;
```

6. Obtener los usuarios que todavía no han pagado su parte correspondiente en un gasto concreto.

	Nombre	Cuota_deuda
▶	Alejandro García	15.00
	Diana Pérez	15.00

```
SELECT
    U.Nombre,
    RG.Cuota_deuda
FROM REPARTO_GASTO RG
JOIN USUARIO U ON RG.Id_Usuario = U.Id_Usuario
WHERE RG.Id_Gasto = 1
    AND RG.Id_Usuario NOT IN (
        SELECT Id_Emisor
```

```

FROM LIQUIDACION
WHERE Id_Grupo = 1
      AND Estado = 'completado'
);

```

7. Calcular el gasto total por mes en un grupo para representarlo en un gráfico de barras mensual.

	Mes	Total_mes
▶	2025-06	425.00

```

SELECT
    DATE_FORMAT(G.Fecha, '%Y-%m') AS Mes,
    SUM(G.Monto_total)             AS Total_mes
FROM GASTO G
WHERE G.Id_Grupo = 1
GROUP BY Mes
ORDER BY Mes ASC;

```

8. Mostrar cuánto dinero ha adelantado en total cada miembro de un grupo pagando gastos.

	Nombre	Total_pagado
▶	Carlos Martínez	320.00
	Alejandro García	60.00
	Beatriz López	45.00
	Diana Pérez	0.00

```

SELECT
    U.Nombre,
    COALESCE(SUM(G.Monto_total), 0) AS Total_pagado
FROM MIEMBROS_GRUPO MG
JOIN USUARIO U ON MG.Id_Usuario = U.Id_Usuario
LEFT JOIN GASTO G ON G.Id_Usuario_Pagador = U.Id_Usuario
                AND G.Id_Grupo = MG.Id_Grupo
WHERE MG.Id_Grupo = 1
GROUP BY U.Id_Usuario, U.Nombre
ORDER BY Total_pagado DESC;

```

9. Calcular el balance neto de cada usuario en un grupo, restando lo que debe de lo que ha pagado, para saber quién está en positivo y quién en negativo.

	Nombre	Total_pagado	Total_debe	Balance_neto
▶	Carlos Martínez	320.00	15.00	305.00
	Alejandro García	120.00	37.50	82.50
	Beatriz López	90.00	37.50	52.50
	Diana Pérez	0.00	15.00	-15.00


```

SELECT
    U.Nombre,
    COALESCE(SUM(G.Monto_total), 0) AS Total_pagado,
    COALESCE(SUM(RG.Cuota_deuda), 0) AS Total_debe,
    COALESCE(SUM(G.Monto_total), 0)
    - COALESCE(SUM(RG.Cuota_deuda),0) AS Balance_netto
FROM MIEMBROS_GRUPO MG
JOIN USUARIO U ON MG.Id_Usuario = U.Id_Usuario
LEFT JOIN GASTO G
    ON G.Id_Usuario_Pagador = U.Id_Usuario AND G.Id_Grupo = MG.Id_Grupo
LEFT JOIN REPARTO_GASTO RG
    ON RG.Id_Usuario = U.Id_Usuario AND RG.Id_Gasto IN (
        SELECT Id_Gasto FROM GASTO WHERE Id_Grupo = MG.Id_Grupo
    )
WHERE MG.Id_Grupo = 1
GROUP BY U.Id_Usuario, U.Nombre
ORDER BY Balance_netto DESC;

```

10. Obtener el historial completo de movimientos de un usuario, combinando los gastos que ha pagado y las liquidaciones que ha emitido, ordenado por fecha descendente.

	Tipo	Concepto	Importe	Fecha	Grupo
▶	Gasto	Cena restaurante	60.00	2025-06-01	Viaje Cantabria

```

SELECT
    'Gasto' AS Tipo,
    G.Concepto,
    G.Monto_total AS Importe,
    G.Fecha AS Fecha,
    GR.Titulo AS Grupo
FROM GASTO G
JOIN GRUPO GR ON G.Id_Grupo = GR.Id_Grupo
WHERE G.Id_Usuario_Pagador = 1

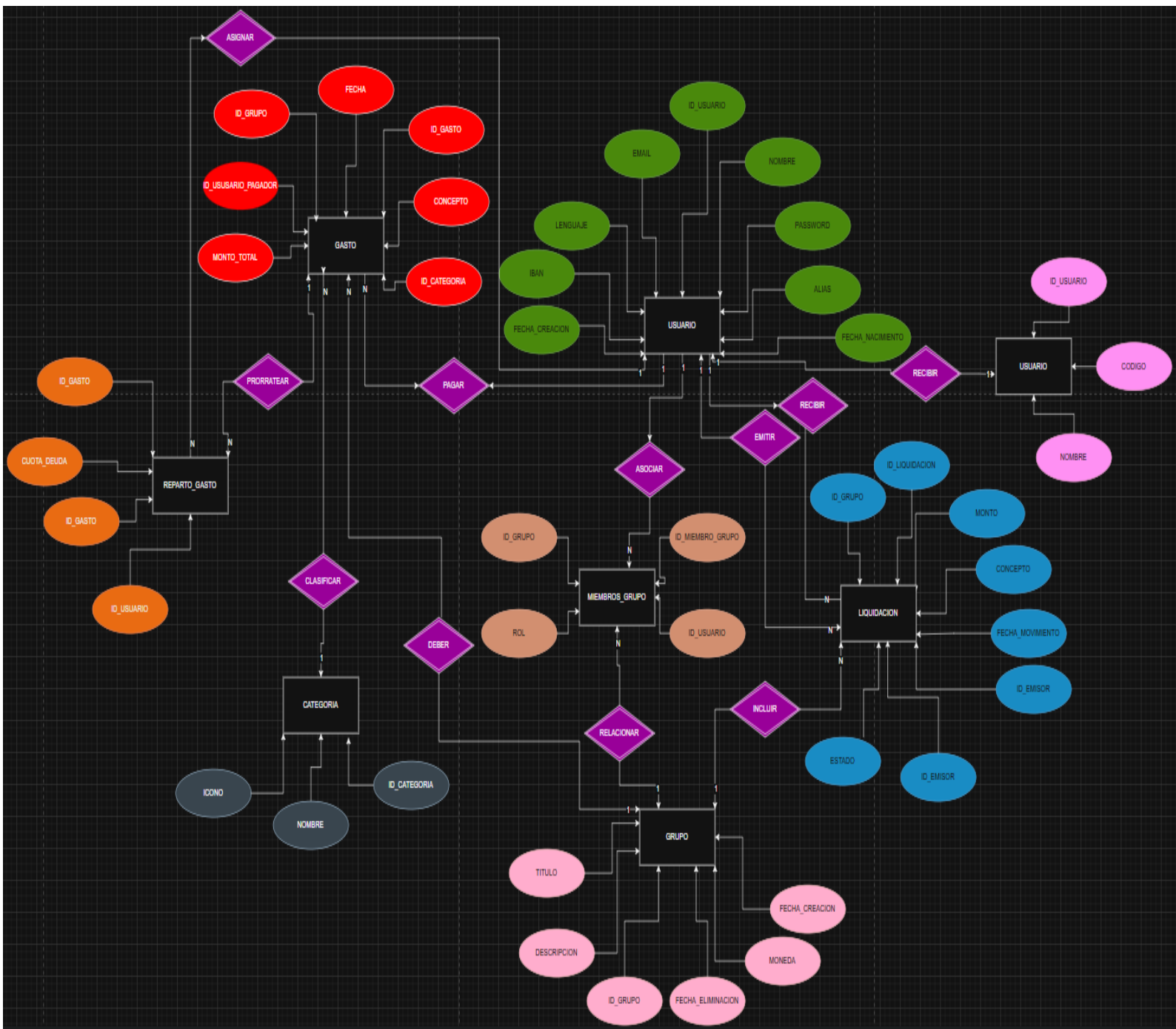
UNION ALL

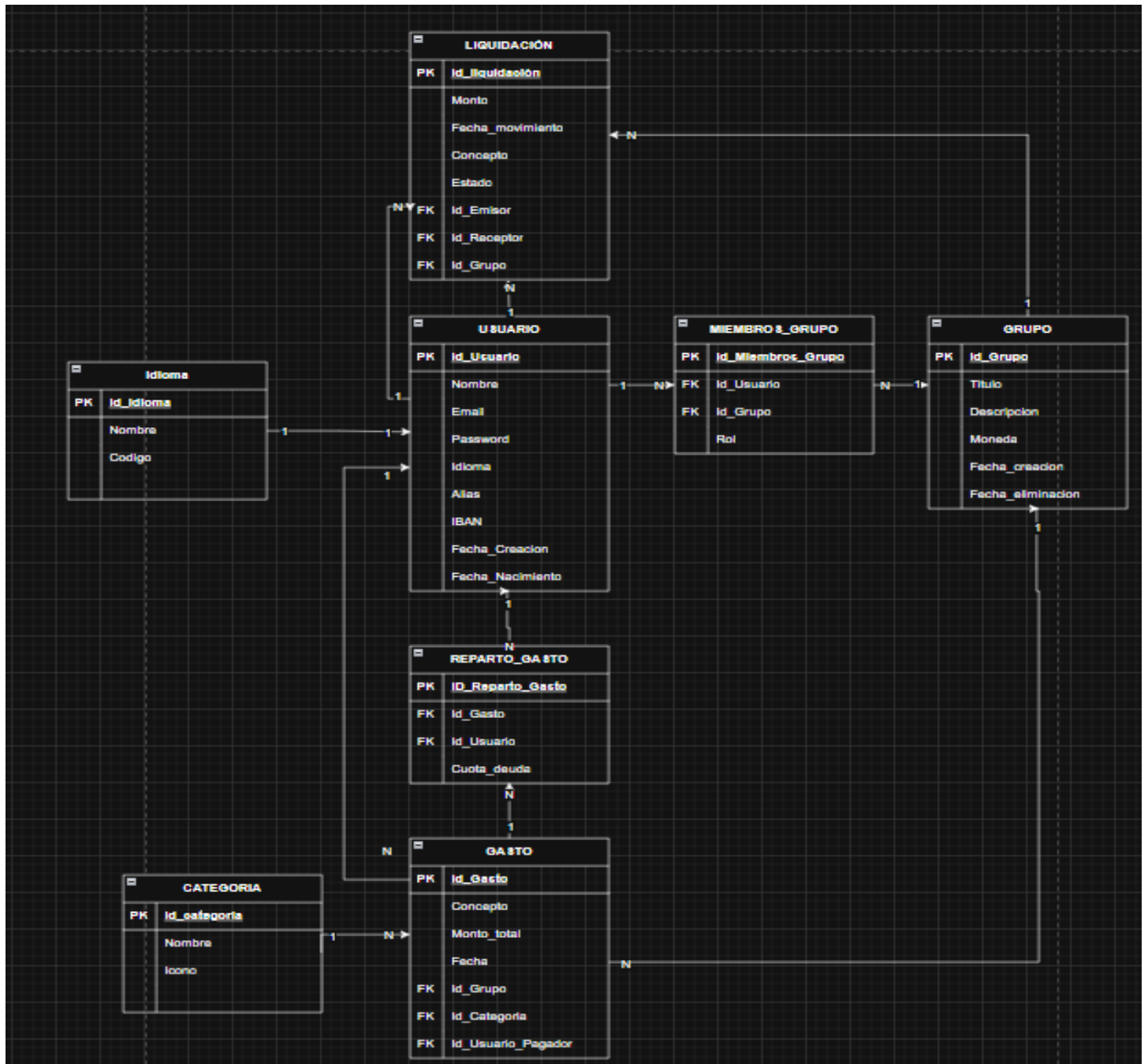
SELECT
    'Liquidacion' AS Tipo,
    L.Concepto,
    L.Monto AS Importe,

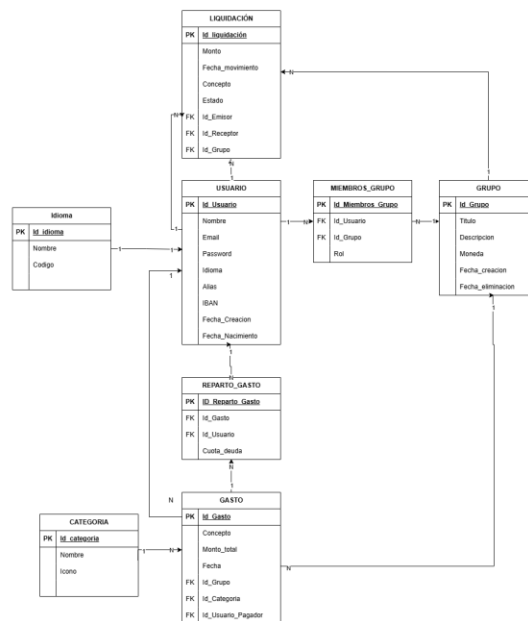
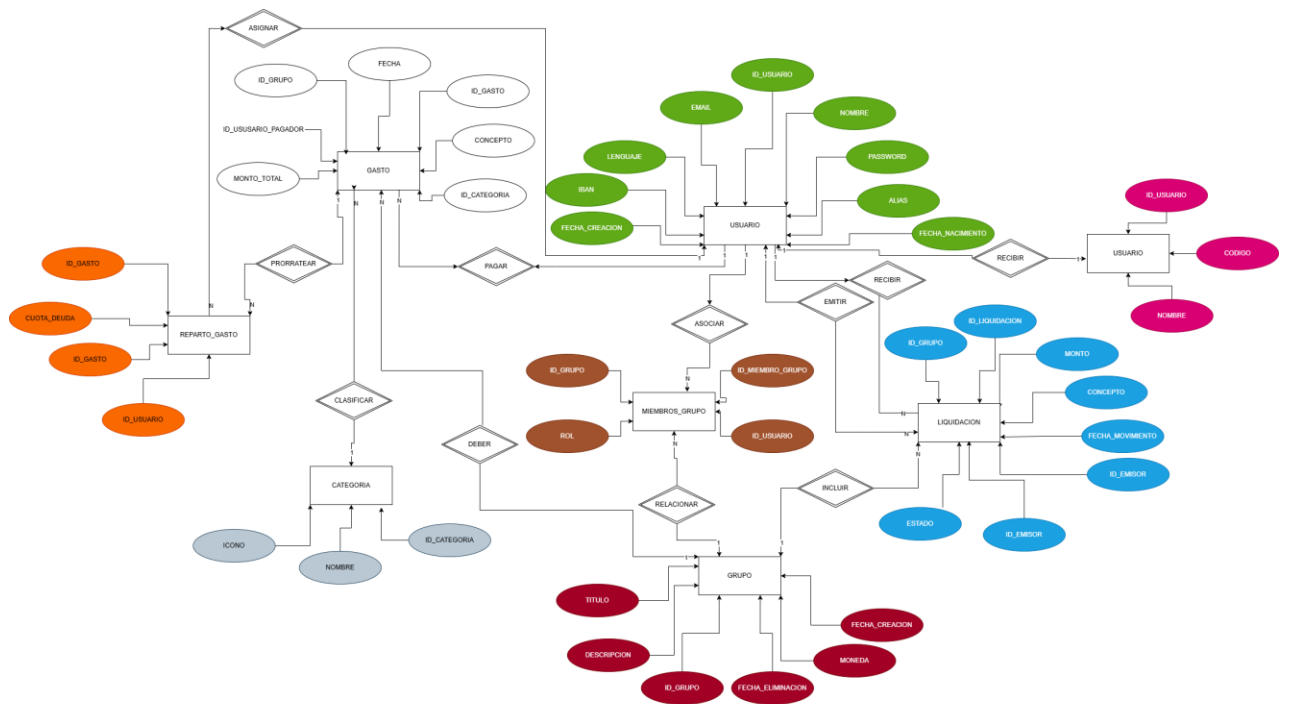
```

```
DATE(L.Fecha_movimiento) AS Fecha,  
GR.Titulo AS Grupo  
FROM LIQUIDACION L  
JOIN GRUPO GR ON L.Id_Grupo = GR.Id_Grupo  
WHERE L.Id_Emisor = 1  
  
ORDER BY Fecha DESC;
```

IMÁGENES







CONCLUSION

Con este proyecto hemos aprendido a crear una aplicación completa desde cero, algo que al principio nos parecía muy difícil. Hemos practicado todo lo que hemos dado en clase, como crear bases de datos en MySQL, conectarlas con Java usando JDBC y diseñar las pantallas con JavaFX. También hemos aprendido a trabajar en equipo, a repartir las tareas y a solucionar los errores que nos han ido surgiendo. Estamos contentos con el resultado porque hemos conseguido hacer una aplicación funcional y creemos que nos ha servido mucho para afianzar los conocimientos del curso.